



21. Python: Exception Handling

"Errors should never pass silently. Unless explicitly silenced."

Previous Section:

[!\[\]\(c3d993ca47bfe2a953c700506ce31fa0_img.jpg\) 20. Python: File Handling](#)

Contents:

[1. The `try-except` Method](#)

[2. The `raise` Method](#)

[3. The `assert` Method](#)

[4. Common Error Types](#)

[Summary](#)

[References](#)

No matter how carefully we write programs, errors will eventually happen. A user enters text instead of a number. A file does not exist. A list index goes out of range. A calculation divides by zero.

And when these problems occur, Python reacts immediately by producing an exception.

At first, exceptions may feel annoying. Programs suddenly stop. Red error messages appear everywhere. The entire application crashes because of one mistake.

But here's the important realization: Exceptions are not enemies. They are warning systems. They tell us something went wrong before the problem becomes even worse.

Imagine if Python ignored every mistake silently:

- Banking systems could calculate incorrect balances
- Medical software could produce wrong results
- AI systems could fail without anyone noticing
- Games could corrupt save files permanently

Errors exist because reliability matters. This is why exception handling is extremely important in programming.

Instead of letting programs crash instantly, Python allows us to detect errors, handle them safely, display meaningful messages, prevent total program failure, recover gracefully

In other words: Exception handling transforms fragile programs into resilient ones. And once you understand how exceptions work, your programs stop behaving like small classroom scripts, and start behaving like real software.

1. The `try-except` Method

The most common way to handle errors in Python is through the `try-except` structure.

Basic syntax:

```
try:
    # risky code
except:
    # error handling
```

Python attempts to execute the code inside the `try` block. If an error occurs, Python immediately jumps to the matching `except` block.

- Example: Handling Multiple Errors

```

try:
    number = int(input("Enter a number: "))
    result = 10 / number

except ValueError:
    print("Error: Please enter a valid number.")

except ZeroDivisionError:
    print("Error: Cannot divide by zero.")

else:
    print("Success! Result is:", result)

```

Possible outputs:

Enter a number: abc

Error: Please enter a valid number.

Enter a number: 0

Error: Cannot divide by zero.

Enter a number: 2

Success! Result is: 5.0

Understanding What Happens - The program performs several risky operations:

```

number = int(input())
result = 10 / number

```

This may produce a `ValueError` if the user enters invalid text. And: `10 / number` may produce a `ZeroDivisionError` if the number is zero.

Instead of crashing, the program handles each error separately. This is one of the biggest advantages of exception handling: Different errors can have different responses.

Printing the Actual Error - Sometimes we want to see the exact error message generated by Python. We can capture the exception object using:

```
except Exception as e
```

- Example:

```
try:
    number = int(input("Enter a number: "))
    result = 10 / number

except Exception as e:
    print("An error occurred:")
    print(e)
    print(type(e))
```

Possible output:

Enter a number: hello

An error occurred:

invalid literal for int() with base 10: 'hello'

<class 'ValueError'>

Here: `e` stores the actual error object; `type(e)` shows the exception type. This is extremely useful for debugging.

Difference Between `else` and `finally`

Many beginners confuse these two blocks. But they behave very differently.

- The `else` Block: It blocks only executes if no exception occurs. Example:

```
try:
    number = int(input("Enter a number: "))

except ValueError:
    print("Invalid input.")

else:
    print("Valid number entered.")
```

If an error happens, `else` is skipped.

- The `finally` Block: It always executes. Whether an error happens, no error happens, the program returns early, `finally` still runs.
- Example:

```
try:
    file = open("data.txt", "r")

except FileNotFoundError:
    print("File not found.")

finally:
    print("Execution completed.")
```

File not found.

Execution completed.

This makes `finally` useful for closing files, releasing resources, database cleanup, network disconnection

2. The `raise` Method

Sometimes Python exceptions are not enough. We may want to create our own errors intentionally. This is where the `raise` keyword becomes useful.

Basic Syntax:

```
raise Exception("Something went wrong")
```

This manually triggers an exception.

- Example: Preventing Negative Age

```
age = int(input("Enter your age: "))

if age < 0:
    raise ValueError("Age cannot be negative.")

print("Age accepted.")
```

Enter your age: -5
 ValueError: Age cannot be negative.

Instead of silently accepting invalid data, the program explicitly rejects it.

Triggering Custom Errors — Better Than Complex `if-else`

Without exceptions, programs often become messy:

```
if score < 0:
    print("Invalid")
else:
    print("Accepted")
```

But with exceptions:

```
score = int(input("Enter score: "))

if score < 0:
    raise ValueError("Score cannot be negative.")

print("Valid score.")
```

This approach is cleaner and more professional. Exceptions separate normal program flow and abnormal situations

Re-Raising Exceptions

Sometimes we catch an exception temporarily, but still want Python to continue raising it afterward.

Example:

```
try:
    result = 10 / 0

except ZeroDivisionError as e:
    print("Logging the error...")
    raise
```

Logging the error...

ZeroDivisionError: division by zero

The error is caught, processed/logged, raised again

This is common in large applications and debugging systems.

3. The `assert` Method

Another extremely useful tool in exception handling is the `assert` statement.

While `try-except` is mainly used to handle unexpected runtime errors, `assert` is commonly used to validate assumptions during development.

In simpler terms: `assert` checks whether something is true.

- If the condition is true: the program continues normally
- If the condition is false: Python immediately raises an `AssertionError`

This makes `assert` very useful for debugging, validating data, testing functions, checking program logic, verifying calculations

Basic Syntax

```
assert condition

# Or with a custom message:
assert condition, "Error message"
```

- Simple Example

```
number = 10

assert number > 0
print("Valid number")
```

Valid number

Because the condition is true, the program continues normally.

- Example with Failure

```
number = -5

assert number > 0, "Number must be positive"
```

AssertionError: Number must be positive

The condition failed, so Python immediately stopped the program.

Using `assert` for Validation

One major use of assertions is validating input or program state. Example:

```
age = int(input("Enter your age: "))
assert age >= 0, "Age cannot be negative"

print("Valid age")
```

Enter your age: -2

AssertionError: Age cannot be negative

This prevents invalid values from silently entering the program.

Using `assert` to Test Functions

Assertions are heavily used when testing functions. Example:


```
def add(a, b):  
    return a + b  
  
assert add(2, 3) == 5  
  
print("Function works correctly")
```

Function works correctly

The assertion verifies whether the function behaves correctly.

Detecting Incorrect Function Results

```
def multiply(a, b):  
    return a + b  
  
assert multiply(2, 3) == 6, "Function calculation is incorrect"
```

AssertionError: Function calculation is incorrect

The function contains a logical mistake: `return a + b` instead of `return a * b`

The assertion catches the issue immediately. This is why assertions are powerful for debugging and testing.

Assertions vs `if-else`

Without assertions:

```
if temperature < 0:  
    print("Invalid temperature")
```

With assertions:

```
assert temperature >= 0, "Temperature cannot be negative"
```

Assertions are cleaner when validating conditions that should never logically fail.

Important Note About Assertions

Assertions are mainly intended for debugging, testing, development validation. They are not always ideal for handling normal user input in production systems. For example: invalid passwords, missing forms, user mistakes are usually better handled using: `try-except`, or regular validation logic. Assertions are best for checking programmer assumptions.

- Example Combining `try-except` and `assert`

```
try:
    score = int(input("Enter score: "))

    assert score >= 0, "Score cannot be negative"
    print("Valid score")

except ValueError:
    print("Please enter a valid number")

except AssertionError as e:
    print(e)
```

```
Enter score: -10
Score cannot be negative
```

This combines input validation, exception handling, assertions, together into one safer system.

4. Common Error Types

Below are some of the most common Python exceptions.

Exception	Description	Example
<code>AssertionError</code>	Raised when the assert statement fails.	<code>assert False</code>
<code>AttributeError</code>	Raised when attribute reference fails.	<code>"abc".append(1)</code>
<code>EOFError</code>	Raised when input() hits end-of-file.	<code>input()</code>
<code>FloatingPointError</code>	Raised when floating point operation fails.	<code>import math</code>
<code>GeneratorExit</code>	Raised when a generator is closed.	<code>generator.close()</code>
<code>ImportError</code>	Raised when module is not found.	<code>import unknownmodule</code>
<code>IndexError</code>	Raised when sequence index is out of range.	<code>[1,2][5]</code>
<code>KeyError</code>	Raised when dictionary key is missing.	<code>{"a":1}["b"]</code>
<code>KeyboardInterrupt</code>	Raised when user interrupts program.	<code>Ctrl + C</code>
<code>MemoryError</code>	Raised when memory runs out.	Very large allocations
<code>NameError</code>	Raised when variable is undefined.	<code>print(x)</code>
<code>NotImplementedError</code>	Raised by unfinished methods.	<code>raise NotImplementedError</code>
<code>OSError</code>	Raised for operating system errors.	<code>open("/invalid/path")</code>
<code>OverflowError</code>	Raised when arithmetic result is too large.	<code>import math; math.exp(1000)</code>
<code>ReferenceError</code>	Raised for invalid weak references.	Weak reference access
<code>RuntimeError</code>	Generic runtime-related error.	<code>raise RuntimeError()</code>
<code>StopIteration</code>	Raised when iterator has no more values.	<code>next(iter([]))</code>
<code>SyntaxError</code>	Raised when syntax is invalid.	<code>if True print("Hi")</code>
<code>IndentationError</code>	Raised for incorrect indentation.	Improper spacing

Exception	Description	Example
<code>TabError</code>	Raised for inconsistent tabs/spaces.	Mixed indentation
<code>SystemError</code>	Raised for internal interpreter issues.	Internal Python issue
<code>SystemExit</code>	Raised by <code>sys.exit()</code> .	<code>import sys; sys.exit()</code>
<code>TypeError</code>	Raised for invalid object type usage.	<code>"2" + 2</code>
<code>UnboundLocalError</code>	Raised for local variable access before assignment.	Access before assignment
<code>UnicodeError</code>	Raised for Unicode-related issues.	Invalid encoding
<code>UnicodeEncodeError</code>	Raised during encoding failure.	<code>"é".encode("ascii")</code>
<code>UnicodeDecodeError</code>	Raised during decoding failure.	<code>b'\xff'.decode('utf-8')</code>
<code>UnicodeTranslateError</code>	Raised during translation failure.	Unicode translation issue
<code>ValueError</code>	Raised for correct type but invalid value.	<code>int("abc")</code>
<code>ZeroDivisionError</code>	Raised when dividing by zero.	<code>10 / 0</code>

Logical Errors

Not all errors produce exceptions. Some programs run perfectly fine, but still produce incorrect results. These are called logical errors. Example:

```
radius = 5

area = 2 * 3.14 * radius * radius
print(area)
```

The program runs successfully. No exception occurs. But the formula is wrong because: $\text{Area} = \pi r^2$ not: $2\pi r^2$

Logical errors are often harder to detect because Python does not know your intended logic. To Python: valid syntax = acceptable; correct mathematics = your responsibility. And this is why debugging becomes an essential programming skill.

Summary

Python programs inevitably produce errors because no program is perfect. Users make mistakes. Files go missing. Data becomes corrupted. Unexpected situations occur constantly.

But exception handling allows programs to respond intelligently instead of crashing immediately. Using `try` , `except` , `else` , `finally` , `raise` , and `assert` , we can build programs that are safer, cleaner, more professional, easier to debug, more resilient

And perhaps the most important lesson is this: ***Good programmers are not people who never create errors. Good programmers are people who know how to handle them properly.***

References

<https://docs.python.org/3/tutorial/errors.html>

<https://www.geeksforgeeks.org/python/python-exception-handling/>

Next Section:

</> [22. Python: Libraries](#)